Employee Account Lifecycle Automation

A Python tool that keeps a user directory in step with HR events (joiners, movers, leavers)

Week 3 - Software Development and Automation Implementation | Yuva Virtual Internship - IT Support | Author: **Elyas Ghwiet** | Run dates in the evidence: 1 Nov - 20 Dec 2026 (simulated)

Executive summary

The problem. In a company, IT support must create an account for every new employee, move people to the right access groups when they change department, and switch accounts off the day someone leaves. Done by hand this is slow, inconsistent and risky: a forgotten leaver keeps access to company systems, and a typo in a username or group causes support tickets.

What I built. `account_automation.py` is a command-line tool written in Python (standard library only). It reads a CSV file of HR events (HIRE, TERMINATE, TRANSFER), checks every row, and applies the changes to a simulated user directory. It creates unique usernames (Arabic names are transliterated), assigns groups from a department policy, generates salted-and-hashed temporary passwords, disables leavers, and writes an audit log, a readable report and a credentials file on every run.

Why it is reliable. Running the same file twice changes nothing (idempotent). `--dry-run` previews a run. Bad rows are rejected with a clear reason while the rest of the file is still processed. A corrupt directory file is never overwritten. Exit codes 0 / 1 / 2 let a scheduler or script know what happened.

Results (all from real runs on my computer). 41 of 41 automated tests passed (Figure 15); 14 execution screenshots cover the normal flow, idempotent re-runs, transfer/terminate, bad input, fatal errors and the 30-day purge notice; 5 defects found by my own tests and runs were fixed before submission. Total effort: **35 hours** (limit 30-35 h).

How this report maps to the grading criteria

Criterion	Where to find the evidence
Functionality and reliability of the script	Section 4 (code explained with snippets); Section 5.2 test cases TC-01 to TC-12 with screenshots; idempotency in Figure 9.
Originality and creativity	Section 7: Arabic-name transliteration with override, future-dated (deferred) events, safe leaver handling, re-hire logic, 30-day purge, atomic saves.
Depth of documentation and clarity	Executive summary above; Sections 2-4 (research, design, explanation); two diagrams; appendices with the complete code, tests and data.
Quality of testing and error handling	Section 5: 41 automated tests by category, 12 manual test cases with screenshots, and Section 6 (defects found and fixed).
Adherence to the 30-35 hour guideline	Section 9: time log with 7 + 13 + 4 + 11 = 35 hours.

1. Problem statement and goals

Every IT help desk handles the same three events again and again. A **joiner** needs an account, a unique username, the right group memberships and a first password. A **mover** (someone who changes department) must lose the access of the old role and receive the access of the new one. A **leaver** must be disabled quickly, because an active account of a former employee is a classic security gap. Frameworks such as CIS Controls and NIST SP 800-53 treat this account life cycle as a basic control (Section 2).

The task of Week 3 was to design and implement a small software solution that automates a repetitive IT job. I chose account provisioning because it is realistic, it can be tested safely without administrator rights on a real server, and it forces careful thinking about errors, because a wrong action affects real people.

Goals I set before writing code

- **Correct:** the directory must always match the HR file, including edge cases (duplicates, unknown employees, re-hires, same-day events).
- **Safe:** never destroy data; never store plain-text passwords in the directory; never crash on a bad row.
- **Repeatable:** same input twice gives the same state (idempotent), and I can preview with `--dry-run`.
- **Traceable:** every decision is recorded in an append-only audit log and in a report.
- **Replaceable:** all directory access is in one class so a real back end (Active Directory, LDAP) can replace the JSON file later.

Scope and limits. The tool works on a simulated directory (a JSON file). It does not talk to a real Active Directory; this is a deliberate choice so that it can be tested by anyone (Section 8).

2. Initial research

Before designing the tool I read the following sources (7 hours including reading Python documentation). The table shows what each source says and how it changed my design. I only list statements that I checked on the source page.

Source	What it says (checked)	Effect on my design
CIS Controls v8, Safeguard 5.3 - Disable Dormant Accounts	"Delete or disable any dormant accounts after a period of 45 days of inactivity, where supported." (asset type: Users, function: Respond)	Leavers are **disabled first, not deleted**; deletion only after a retention period (30 days in `policy.json`) and only with `--purge`.
NIST SP 800-53 Rev. 5, control AC-2(3) Disable Accounts	Accounts must be disabled within an organisation-defined time when they are expired, no longer associated with a user, violate policy, or are inactive.	TERMINATE disables on the effective date, removes all groups and invalidates the password in the same step.
OWASP Password Storage Cheat Sheet	A unique salt per password; PBKDF2-HMAC-SHA256 with 600,000 iterations recommended.	I use PBKDF2-HMAC-SHA256 with a random salt per password. I chose **200,000** rounds so tests run fast; for production I would raise it to at least 600,000 (listed in Section 8).
NIST SP 800-63B (June 2017 revision), 5.1.1.2	Memorized secrets shall be salted and hashed with a one-way key derivation function; salt at least 32 bits; PBKDF2 typically at least 10,000 iterations. Verifiers should not impose arbitrary composition rules on user-chosen passwords.	My 16-hex-character salt is 64 bits; 200,000 rounds is above the NIST minimum. The composition rule (upper, lower, digit, symbol) applies only to the **machine-generated temporary** password, not to what the user later chooses.

Source	What it says (checked)	Effect on my design
Python documentation: `hashlib`, `secrets`, `csv`, `argparse`, `tempfile`, `os.replace`	`secrets` is meant for security tokens; `hashlib.pbkdf2_hmac` is in the standard library; `os.replace` is an atomic rename.	No third-party packages are needed; `secrets` for passwords and salts; temp file + `os.replace` for crash-safe saves.

Note: NIST SP 800-63B was superseded by SP 800-63-4 in August 2025; I use the 2017 text only for the salt and key-derivation facts above.

3. Design

3.1 Architecture and data flow

A run has five stages, shown in Figure 1. The important design idea is the separation of two kinds of failure. A **RowError** concerns one HR row (for example an unknown department); it is recorded as REJECTED and the run continues. A **FatalError** concerns the whole run (for example a missing file or a corrupt directory); the run stops, exits with code 2, and leaves existing data untouched.

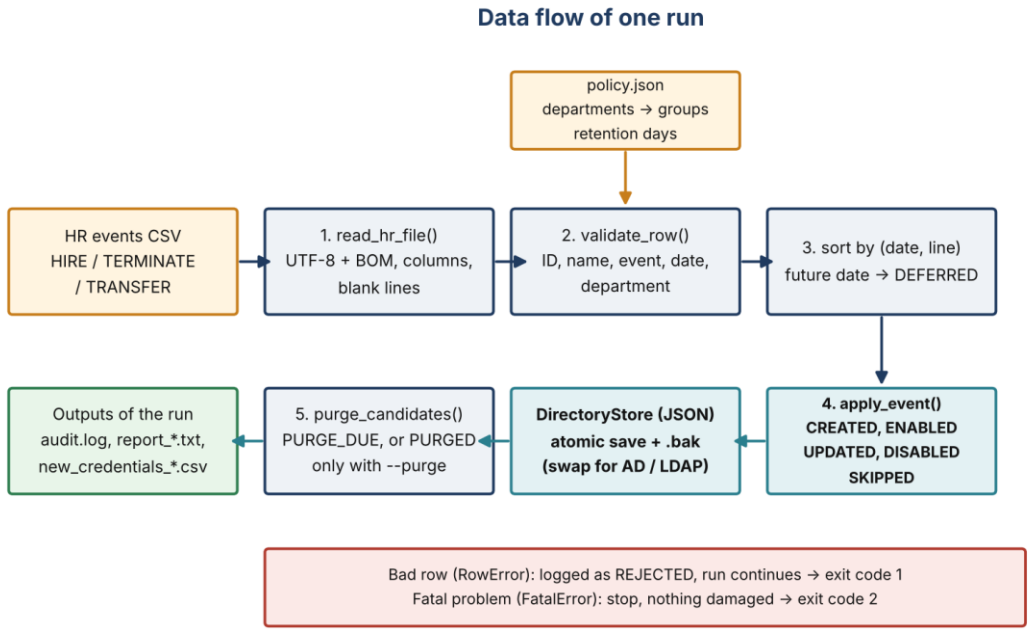


Figure 1. *Data flow of one run. Orange = inputs, teal = directory logic, green = outputs, red = error paths.*

3.2 Data model

File	Format	Purpose
HR events file	CSV: `employee_id, full_name, department, event, effective_date` (+ optional `latin_name`)	Input. One row = one event. `latin_name` lets HR give the Latin spelling of an Arabic name.
`data/policy.json`	JSON: departments -> groups, `default_groups`, `retention_days`	Business rules separated from code: IT = it-staff, remote-access, admin-tools; HR = hr-users, payroll-readonly; Finance = finance-users, erp-access; Sales = sales-users, crm-access; everyone also gets all-staff.
`data/directory.json`	JSON: `users` keyed by username	The simulated directory: employee_id, full_name, department, status (active / disabled), groups, created, disabled_on, password_hash, must_change_password.
`output/audit.log`	Text, append-only	One line per decision with timestamp, status, event, employee, username and detail.

File	Format	Purpose
`output/report_<stamp>.txt`	Text	Human-readable summary and details of one run.
`output/new_credentials_<stamp>.csv`	CSV, file mode 600	Temporary passwords, only for newly created or re-enabled accounts. Must be handed over securely, then deleted.

3.3 Rules for each event

Event	Condition	Result	Status
HIRE	no account yet	Create account, unique username, department + default groups, hashed temporary password, `must_change_password = true`	CREATED
HIRE	account active	Nothing changes (this makes re-running safe)	SKIPPED
HIRE	account disabled	Re-enable the same username, new groups and new temporary password	ENABLED
TERMINATE	account active	status = disabled, groups removed, password hash cleared, `disabled_on` recorded	DISABLED
TERMINATE	already disabled	Nothing changes	SKIPPED
TERMINATE / TRANSFER	unknown employee	Row refused with reason	REJECTED
TRANSFER	active, other department	Replace groups with the new department's groups	UPDATED
TRANSFER	same department	Nothing changes	SKIPPED
TRANSFER	account disabled	Refused: cannot move a disabled account	REJECTED
any	effective date after run date (`--as-of`)	Not applied yet; applied on a later run	DEFERRED
(purge)	disabled longer than `retention_days`	Reported; deleted only if `--purge` is given	PURGE_DUE / PURGED

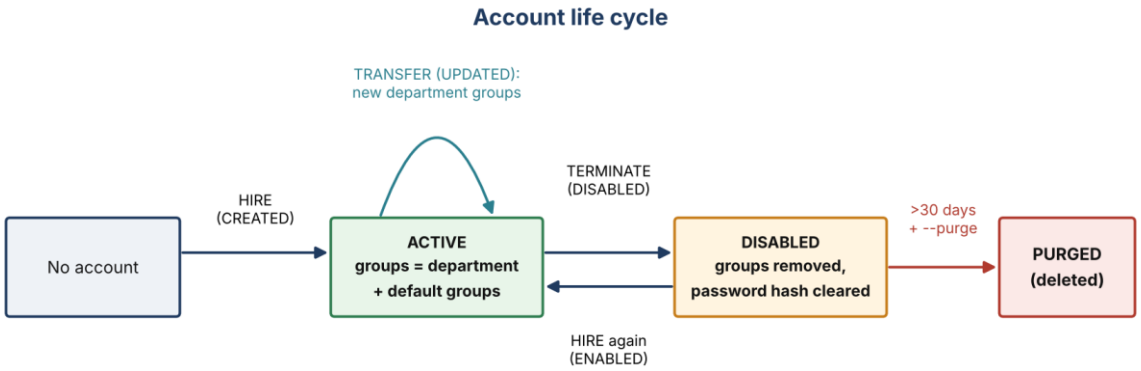


Figure 2. *Account life cycle implied by the rules above.*

3.4 Decision log (alternatives I considered)

Question	Decision	Why / alternative rejected
How to name users?	First initial + surname, ASCII, max 20 characters; add 2, 3, ... on collision	Common corporate convention. Alternative "full name" is long and hard to type. Names of disabled accounts stay reserved so a username is never reused by another person (audit trail).

Question	Decision	Why / alternative rejected
Arabic names?	Transliteration table + optional `latin_name` column; accents removed with Unicode NFKD	Real HR data contains Arabic names; Windows usernames should be ASCII. HR can override the machine transliteration.
Password storage?	PBKDF2-HMAC-SHA256, random 64-bit salt, stored as `pbkdf2_sha256$rounds$salt$digest`	Plain text or an unsalted hash would be a security defect. bcrypt/argon2 need extra packages; I wanted a standard-library-only tool.
Delete leavers immediately?	No: disable, keep 30 days, purge only with `--purge`	Allows recovery from HR mistakes and keeps an audit trail (CIS 5.3 speaks of deleting **or** disabling).
Order of events?	Sort by (effective date, line number)	Rows in an HR export are not always in time order; a HIRE and TRANSFER on one day must apply in file order.
Future-dated rows?	DEFERRED, applied by a later run	Real HR files contain next month's joiners. Skipping silently would lose them; failing would block the run.
Saving the directory?	Write to a temp file, then `os.replace`; keep `.bak` of the previous version	A crash or full disk during writing can never leave half a JSON file.
Match employees by what?	`employee_id` only	Names change and repeat; IDs are unique. Limitation: a wrong name on a known ID is not detected (Section 8).

4. Implementation: how the code works

The script has about 420 lines and uses only the standard library (`argparse, csv, json, logging, secrets, hashlib, tempfile, unicodedata, dataclasses`). The most important parts are explained below with the real code; the complete listing is in Appendix A.

4.1 Usernames

```
def base_username(name, max_len=USERNAME_MAX):
    words = to_ascii_words(name)
    if not any(ch.isalpha() for w in words for ch in w):
        raise RowError("name has no usable letters for a username")
    first, last = words[0], words[-1]
    return (first[0] + last if len(words) > 1 else first)[:max_len]


def unique_username(base, taken):
    """base, base2, base3, ... - names of disabled accounts stay taken (audit trail)."""
    if base not in taken:
        return base
    n = 2
    while "%s%d" % (base[:USERNAME_MAX - len(str(n))], n) in taken:
        n += 1
    return "%s%d" % (base[:USERNAME_MAX - len(str(n))], n)
```

Listing 1 - base_username and unique_username (lines 90-105).

`to_ascii_words` returns the lower-case ASCII words of the name. A name such as "Sara Al-Harbi" becomes `salharbi` (first letter of the first word + the last word; the hyphen is removed). A single-word name uses the word itself. If nothing alphabetic remains, for example a name of only digits and symbols, the row is **rejected** instead of creating an empty username: this check (`isalpha`) was added after one of my own tests failed (Section 6). `unique_username` appends 2, 3, ... until the name is free; the slice `base[:USERNAME_MAX - len(str(n))]` guarantees that the number never pushes the name above 20 characters. In the evidence, the second Hassan receives `ahassan2` (`ahassan` was taken; Figure 5).

4.2 Passwords

```
def new_temp_password(length=14):
    alphabet = string.ascii_letters + string.digits + "!@#$%&*"
    while True:
        pw = "".join(secrets.choice(alphabet) for _ in range(length))
        if (any(c.islower() for c in pw) and any(c.isupper() for c in pw)
                and any(c.isdigit() for c in pw) and any(c in "!@#$%&*" for c in pw)):
            return pw


def hash_password(pw):
    salt = secrets.token_hex(8)
    digest = hashlib.pbkdf2_hmac("sha256", pw.encode(), salt.encode(), PBKDF2_ROUNDS).hex()
    return "pbkdf2_sha256$%d$%s$%s" % (PBKDF2_ROUNDS, salt, digest)
```

Listing 2 - temporary password and its hash (lines 109-121).

`secrets.choice` uses the operating system's secure random generator (unlike `random`). The loop repeats until the 14-character password contains lower case, upper case, a digit and a symbol. Only the hash goes into the directory: `pbkdf2_hmac('sha256', ...)` with a fresh random salt and 200,000 rounds. The clear-text password exists in memory and in the credentials file only, and the file is created with mode 600 where the operating system supports it. Every new account has `must_change_password = true`.

4.3 Validating one HR row

```
def validate_row(row, policy):
    employee_id = row["employee_id"]
    if not EMPLOYEE_ID_RE.match(employee_id):
        raise RowError("invalid employee_id %r (3-20 letters, digits or '-')" % employee_id)
    if not row["full_name"] or len(row["full_name"]) > 80:
        raise RowError("full_name is empty or longer than 80 characters")
    event = row["event"].upper()
    if event not in EVENTS:
        raise RowError("unknown event %r (expected %s)" % (row["event"], "/".join(EVENTS)))
    try:
        eff = datetime.strptime(row["effective_date"], "%Y-%m-%d").date()
    except ValueError:
        raise RowError("effective_date %r is not in YYYY-MM-DD format" % row["effective_date"])
    dept = policy["_dept_lookup"].get(row["department"].lower())
    if dept is None:
        raise RowError("unknown department %r (policy knows: %s)"
                       % (row["department"], ", ".join(sorted(policy["departments"]))))
    return {"employee_id": employee_id, "full_name": row["full_name"], "event": event,
            "effective": eff, "department": dept, "latin_name": row.get("latin_name", "")}
```

Listing 3 - validate_row (lines 217-235).

Each rule raises `RowError` with a sentence that tells HR what to fix, for example `unknown department 'Legal' (policy knows: Finance, HR, IT, Sales)`. Department matching is case-insensitive through a lookup table built in `load_policy`. The event name is converted to upper case, so `hire` works too.

4.4 Applying events

```
    if action == "HIRE":
        if user and user["status"] == "active":
            return "SKIPPED", username, "already has an active account"
        pw = new_temp_password()
        if user:                                                # re-hire of a disabled account
            user.update(status="active", department=dept, groups=groups_for(policy, dept),
                        disabled_on=None, password_hash=hash_password(pw), must_change_password=True)
            credentials.append((username, pw))
            return "ENABLED", username, "re-hire: account re-enabled in %s" % dept
        base = base_username(ev["latin_name"] or ev["full_name"])
        username = unique_username(base, set(store.users))
        store.users[username] = {
```

```
            "employee_id": ev["employee_id"], "full_name": ev["full_name"], "department": dept,
            "status": "active", "groups": groups_for(policy, dept), "created": as_of.isoformat(),
            "disabled_on": None, "password_hash": hash_password(pw), "must_change_password": True}
        credentials.append((username, pw))
        note = "" if username == base else " (%s was taken)" % base
        return "CREATED", username, "department %s, groups %s%s" % (dept, ",
".join(store.users[username]["groups"]), note)
```

Listing 4 - the HIRE branch of apply_event (lines 244-261).

The first `if` is what makes the script **idempotent**: if the person already has an active account the function returns SKIPPED and changes nothing. The second branch handles a re-hire: the old username is re-enabled instead of creating a duplicate. Only a genuinely new person reaches the creation code.

```
    if action == "TERMINATE":
        if user["status"] == "disabled":
            return "SKIPPED", username, "account already disabled"
        user.update(status="disabled", groups=[], disabled_on=ev["effective"].isoformat(),
                    password_hash=None, must_change_password=False)
        return "DISABLED", username, "groups removed, password invalidated"

    if action == "TRANSFER":
        if user["status"] != "active":
            raise RowError("cannot transfer: account %s is disabled" % username)
        if user["department"] == dept:
            return "SKIPPED", username, "already in %s" % dept
        old = user["department"]
        user.update(department=dept, groups=groups_for(policy, dept))
        return "UPDATED", username, "moved %s -> %s, groups %s" % (old, dept, ",
".join(user["groups"]))
```

Listing 5 - TERMINATE and TRANSFER branches (lines 266-280).

TERMINATE does four things in one `update`: status, group list, `disabled_on` date and password hash, so there is no state in which an account is half-disabled. A disabled account cannot be transferred (REJECTED) because that would silently give access back.

4.5 The run loop

```
    results, credentials, events = [], [], []
    for line, row in rows:
        try:
            ev = validate_row(row, policy)
            ev["line"] = line
            events.append(ev)
        except RowError as exc:
            results.append(Result(line, row.get("employee_id", ""), row.get("event", "").upper(), "REJECTED",
"", str(exc)))

    events.sort(key=lambda e: (e["effective"], e["line"]))        # apply in date order
    for ev in events:
        if ev["effective"] > as_of:
            results.append(Result(ev["line"], ev["employee_id"], ev["event"], "DEFERRED", "",
                                  "effective %s is after run date %s" % (ev["effective"], as_of)))
            continue
        try:
            status, username, detail = apply_event(ev, store, policy, as_of, credentials)
            results.append(Result(ev["line"], ev["employee_id"], ev["event"], status, username, detail))
        except RowError as exc:
            results.append(Result(ev["line"], ev["employee_id"], ev["event"], "REJECTED", "", str(exc)))

    for username in purge_candidates(store, policy, as_of):
        if purge:
            del store.users[username]
            results.append(Result(0, "", "PURGE", "PURGED", username, "disabled longer than %s days" %
policy["retention_days"]))
        else:
            results.append(Result(0, "", "PURGE", "PURGE_DUE", username,
                                  "disabled longer than %s days - run with --purge to delete" %
policy["retention_days"]))
```

Listing 6 - validation, ordering, deferral, apply and purge inside run() (lines 299-326).

The loop first validates all rows (rejections are collected, never raised to the top), then sorts the valid events by `(effective, line)`. An event whose date is after `--as-of` becomes DEFERRED. Finally, accounts disabled longer than the retention period are reported as PURGE_DUE, and only deleted when `--purge` is set. In dry-run mode nothing is written.

4.6 Safe saving

```
    def save(self):
        """Write atomically and keep one backup of the previous version."""
        folder = os.path.dirname(os.path.abspath(self.path))
        os.makedirs(folder, exist_ok=True)
        if os.path.exists(self.path):
            shutil.copy2(self.path, self.path + ".bak")
        fd, tmp = tempfile.mkstemp(dir=folder, suffix=".tmp")
        try:
            with os.fdopen(fd, "w", encoding="utf-8") as f:
                json.dump(self.data, f, indent=2, ensure_ascii=False, sort_keys=True)
            os.replace(tmp, self.path)
        except OSError:
            if os.path.exists(tmp):
                os.remove(tmp)
            raise
```

Listing 7 - DirectoryStore.save (lines 146-160).

The data is written to a temporary file in the same folder and then moved over the real file with `os.replace`, which is atomic on both Windows and Linux. A copy of the previous file is kept as `directory.json.bak`. `load()` (not shown) raises a FatalError with the text "It was NOT modified" when the JSON is corrupt (Figure 13).

4.7 Exit codes and the command line

Option	Meaning
`--hr FILE` (required)	HR events CSV
`--directory`, `--policy`, `--out-dir`	Locations (defaults `data/directory.json`, `data/policy.json`, `output`)
`--as-of YYYY-MM-DD`	Run date. Used for deferral and for the 30-day purge; makes the behaviour testable without waiting
`--dry-run`	Show what would happen, change and write nothing
`--purge`	Delete accounts disabled longer than `retention_days`
Exit code 0 / 1 / 2	All rows fine / at least one row rejected / fatal error

5. Testing

5.1 Automated tests (pytest)

I wrote 41 tests in `tests/test_account_automation.py`. Every test uses temporary folders, so it never touches real data. All 41 pass (Figure 15: `41 passed in 5.36s`, Python 3.14.6, pytest 9.2.0).

Category	Tests	What is checked
Usernames	7	initial + surname; hyphens/apostrophes/accents; Arabic names; single word and 20-char limit; name without letters rejected; collisions numbered; suffix respects the length limit
Passwords	2	14 chars with all four character classes and different each time; only a hash is stored

Category	Tests	What is checked
Hire and idempotency	5	account + groups; `latin_name` beats Arabic name; same username base for two people; future hire deferred then applied; same file twice changes nothing
Terminate / transfer / re-hire / order	7	disable removes access; unknown employee; double terminate; transfer changes groups; transfer of disabled refused; re-hire re-enables same username; events apply in date order
Purge	2	PURGE_DUE reported and `--purge` deletes; recent leaver not due
Validation (parametrised)	6	bad ID, empty name, bad event, bad date, unknown department, ... rejected while the run continues
Dry-run, audit and outputs	3	dry-run changes nothing; audit log, report and credentials written; second run on the same day does not overwrite
File-level and data safety	6	missing HR file; missing column; corrupt directory not overwritten; bad policy; Excel BOM and blank lines; `.bak` backup kept
Command-line exit codes	3	0 when all rows are fine, 1 when a row is rejected, 2 on a fatal error

Total: 7 + 2 + 5 + 7 + 2 + 6 + 3 + 6 + 3 = 41. The full list of test names is in Appendix B.

5.2 Manual test cases with evidence

I also ran the tool by hand in PowerShell inside VS Code. The table lists each case, the expected result and what I actually observed; the screenshots follow in Section 5.3.

ID	Command / input	Expected	Actual	Evidence
TC-01	`--help`	All options documented	As expected	Figure 3
TC-02	`hr_day1.csv --dry-run --as-of 2026-11-01`	6 CREATED, 1 DEFERRED, nothing written	Same; no directory file created	Figure 5
TC-03	same, without `--dry-run`	Accounts created, 3 output files, warning about passwords	Same	Figure 6, Figure 7, Figure 8
TC-04	run `hr_day1.csv` again	Nothing changes	6 SKIPPED, 1 DEFERRED	Figure 9
TC-05	`hr_day2.csv --as-of 2026-11-15`	Transfer, terminate, duplicate hire, unknown ID, deferred hire	UPDATED 1, DISABLED 1, SKIPPED 1, REJECTED 1, DEFERRED 1; exit code 1	Figure 10
TC-06	`hr_bad.csv`	Good row created; each bad row rejected with a reason	CREATED 1, REJECTED 5; exit code 1	Figure 11
TC-07	missing HR file	Clear error, exit code 2	`ERROR HR file not found`; exit 2	Figure 12
TC-08	corrupt `directory.json`	Error, file not overwritten, exit 2	Message "NOT modified"; file unchanged	Figure 13
TC-09	`hr_day2.csv --as-of 2026-12-20`	Deferred hire now applied; leaver due for purge	CREATED `librahim`; PURGE_DUE `salharbi`	Figure 14
TC-10	`pytest -v`	All tests pass	41 passed	Figure 15
TC-11	my own file `my_test.csv` (new directory)	Tool works on data I wrote myself	Re-run gives 3 SKIPPED (idempotent)	Figure 16
TC-12	directory file inspection	Passwords stored only as hashes	`pbkdf2_sha256$200000$...` entries	Figure 8

5.3 Execution screenshots and what each one proves

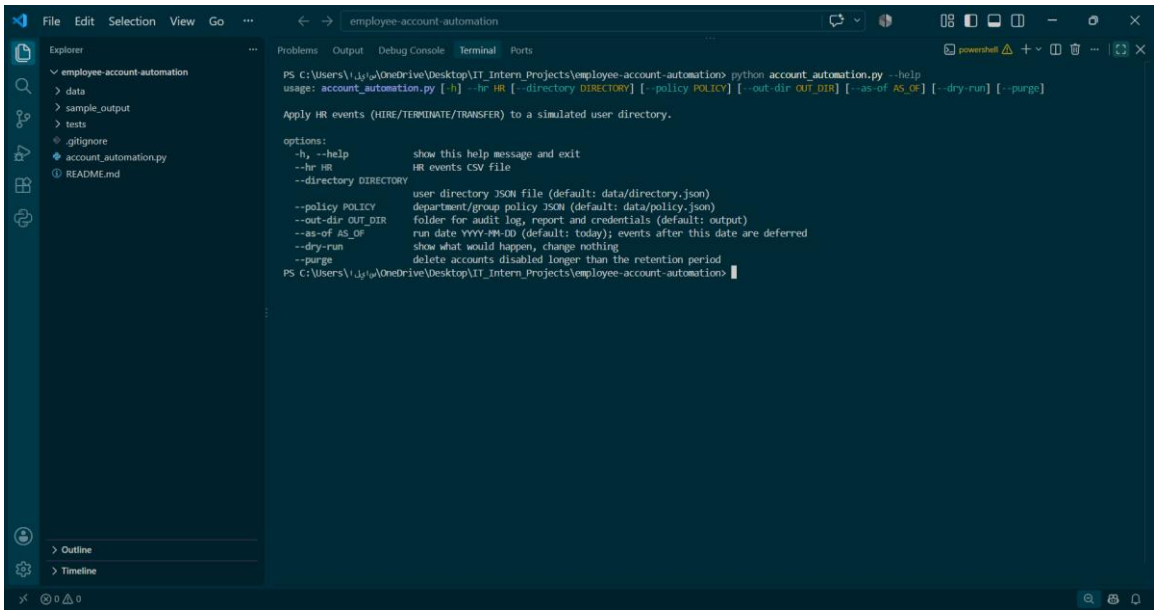


Figure 3. *The tool's help text and project layout in VS Code.*

What it shows. Every option has a description, defaults are shown, and the folder contains `data`, `tests`, `sample_output`, `README.md` and the script. This shows that the tool is documented and usable by someone who has not seen the code.

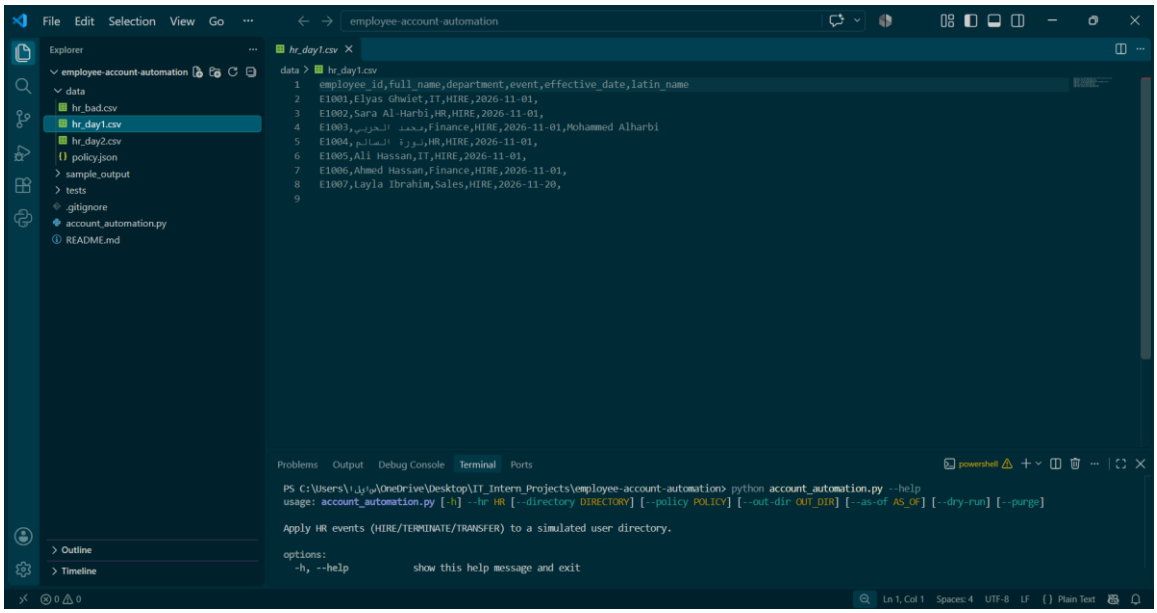


Figure 4. *Input file `hr_day1.csv`.*

What it shows. Seven rows on purpose: two Arabic names (one with a `latin_name` override), two people called Hassan (to force a username collision) and one row with a future date (E1007, 2026-11-20) to test deferral.

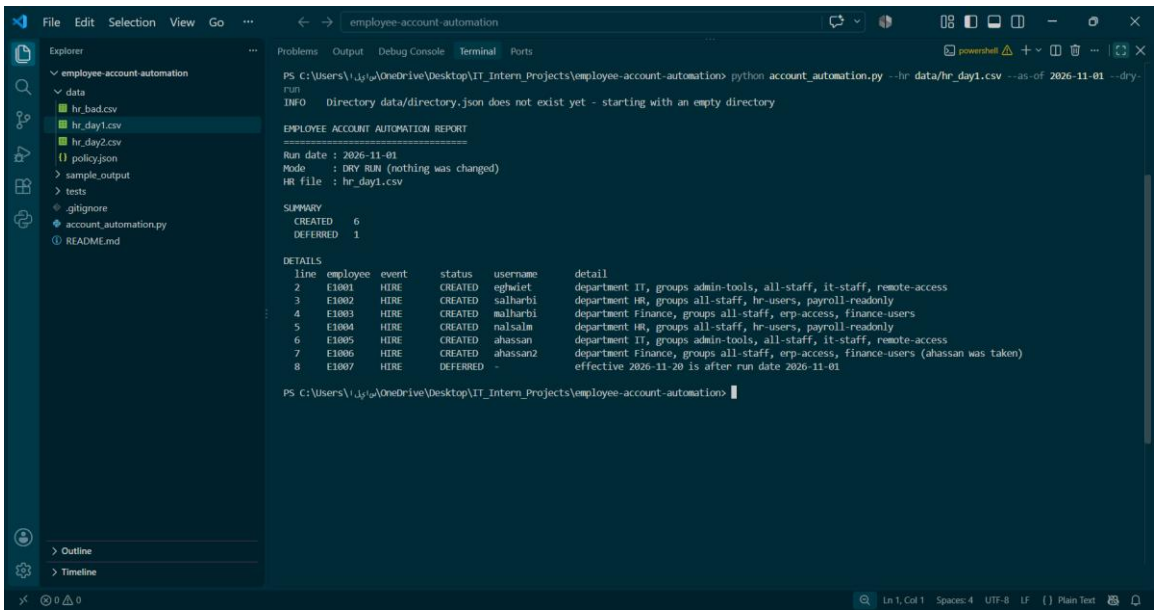


Figure 5. *Dry-run of `hr_day1.csv` with `--as-of 2026-11-01`.*

What it shows. The report says `DRY RUN (nothing was changed)`: 6 CREATED and 1 DEFERRED. The second Hassan became `ahassan2` and the Arabic names became `malharbi` and `nalsalm`. The Explorer shows that no `directory.json` exists yet, so the preview really changed nothing.

```
PS C:\Users\ايلياس\OneDrive\Desktop\IT_Intern_Projects\employee-account-automation> python account_automation.py --hr data/hr_day1.csv --as-of 2026-11-01
INFO    Directory data/directory.json does not exist yet - starting with an empty directory
WARNING Temporary passwords were written to output\new_credentials_2026-11-01_113514.csv - hand them over securely, then delete the file

EMPLOYEE ACCOUNT AUTOMATION REPORT
==================================
Run date : 2026-11-01
Mode     : APPLY
HR file  : hr_day1.csv

SUMMARY
  CREATED    6
  DEFERRED   1

DETAILS
  line  employee  event     status    username      detail
  2     E1001     HIRE      CREATED   eghwiet       department IT, groups admin-tools, all-staff, it-staff, remote-access
  3     E1002     HIRE      CREATED   salharbi      department HR, groups all-staff, hr-users, payroll-readonly
  4     E1003     HIRE      CREATED   malharbi      department Finance, groups all-staff, erp-access, finance-users
  5     E1004     HIRE      CREATED   nalsalm       department HR, groups all-staff, hr-users, payroll-readonly
  6     E1005     HIRE      CREATED   ahassan       department IT, groups admin-tools, all-staff, it-staff, remote-access
  7     E1006     HIRE      CREATED   ahassan2      department Finance, groups all-staff, erp-access, finance-users (ahassan was taken)
  8     E1007     HIRE      DEFERRED  -             effective 2026-11-20 is after run date 2026-11-01
```

Figure 6. *The same file applied for real.*

What it shows. Same results, plus a warning that temporary passwords were written to a credentials file and must be handed over securely and then deleted.

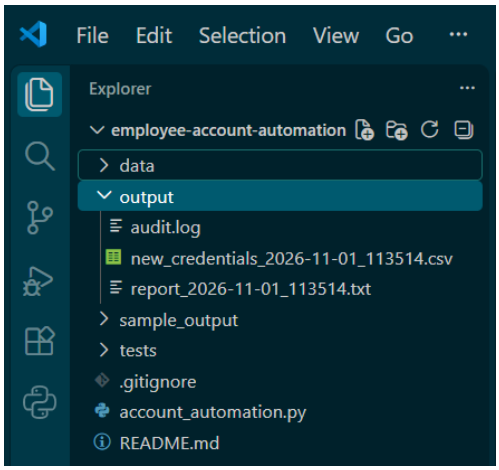


Figure 7. *The `output` folder after the run.*

What it shows. Three artefacts are created automatically: `audit.log`, `report_<date>_<time>.txt` and `new_credentials_<date>_<time>.csv`. The time in the name prevents a later run from overwriting an earlier one (a defect I fixed, Section 6).

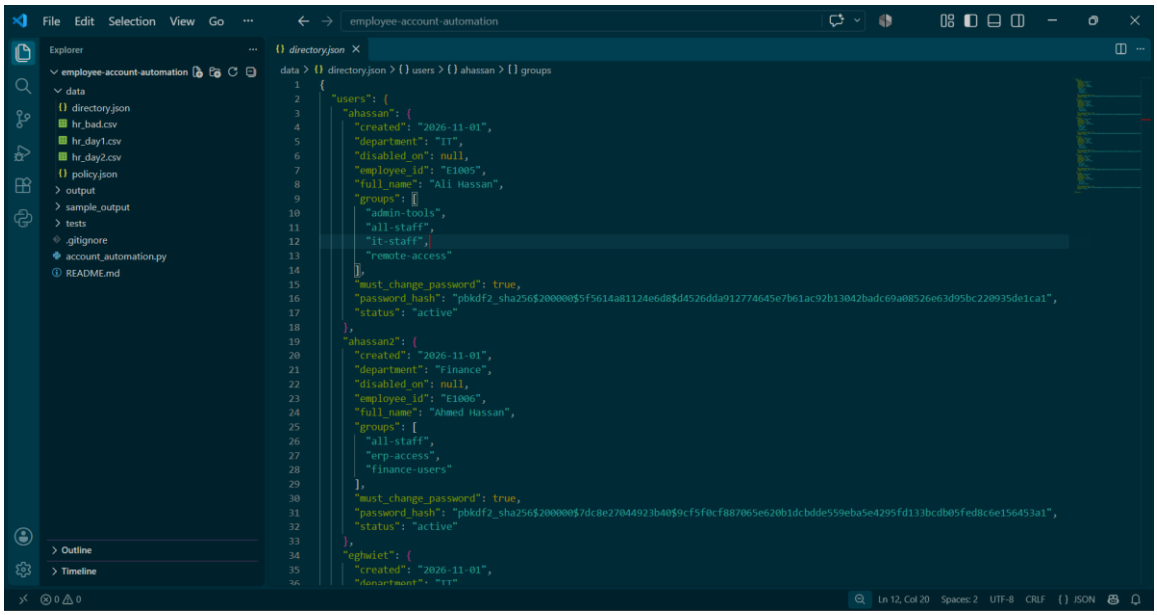


Figure 8. *`directory.json` after the run.*

What it shows. Each user has department, groups, status and `must_change_password: true`. The password field holds a hash in the form `pbkdf2_sha256$200000$salt$digest`, never the password itself.

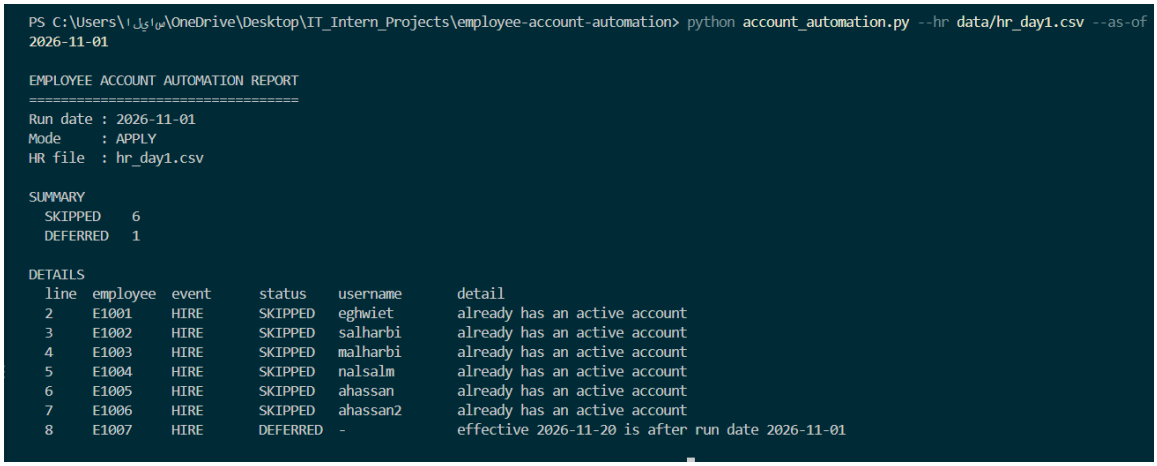


Figure 9. *Second run of the same file (idempotency).*

What it shows. 6 SKIPPED and 1 DEFERRED: running the same HR file twice creates no duplicates and no new passwords. This is the most important property for automation that may be started by a scheduler.

```
PS C:\Users\اسماعيل\OneDrive\Desktop\IT_Intern_Projects\employee-account-automation> python account_automation.py --hr data/hr_day2.csv --as-of
2026-11-15
>> echo $LASTEXITCODE

EMPLOYEE ACCOUNT AUTOMATION REPORT
==================================
Run date : 2026-11-15
Mode     : APPLY
HR file  : hr_day2.csv

SUMMARY
  UPDATED    1
  DISABLED   1
  SKIPPED    1
  DEFERRED   1
  REJECTED   1

DETAILS
  line  employee  event      status    username       detail
  2     E1005     TRANSFER   UPDATED   ahassan        moved IT -> Finance, groups all-staff, erp-access, finance-users
  3     E1002     TERMINATE  DISABLED  salharbi       groups removed, password invalidated
  4     E1001     HIRE       SKIPPED   eghwiet        already has an active account
  5     E1099     TERMINATE  REJECTED  -              no account exists for employee E1099
  6     E1007     HIRE       DEFERRED  -              effective 2026-11-20 is after run date 2026-11-15

WARNING 1 row(s) were rejected - see the report; exit code 1
1
```

Figure 10. *`hr_day2.csv` with `--as-of 2026-11-15`; then `echo $LASTEXITCODE`.*

What it shows. TRANSFER moved `ahassan` from IT to Finance and replaced the groups; TERMINATE disabled `salharbi`; the duplicate HIRE of E1001 was skipped; the unknown employee E1099 was REJECTED with a reason; E1007 stays DEFERRED. The exit code is 1 because one row was rejected.

```
PS C:\Users\اسماعيل\OneDrive\Desktop\IT_Intern_Projects\employee-account-automation> python account_automation.py --hr data/hr_bad.csv --directo
ry data/directory_bad.json --as-of 2026-11-01
>> echo $LASTEXITCODE
INFO    Directory data/directory_bad.json does not exist yet - starting with an empty directory
WARNING Temporary passwords were written to output\new_credentials_2026-11-01_115242.csv - hand them over securely, then delete the file

EMPLOYEE ACCOUNT AUTOMATION REPORT
==================================
Run date : 2026-11-01
Mode     : APPLY
HR file  : hr_bad.csv

SUMMARY
  CREATED    1
  REJECTED   5

DETAILS
  line  employee  event      status    username       detail
  2     E2001     HIRE       CREATED   okhaled        department IT, groups admin-tools, all-staff, it-staff, remote-access
  3     E2002     HIRE       REJECTED  -              unknown department 'Legal' (policy knows: Finance, HR, IT, Sales)
  4     E2003     HIRE       REJECTED  -              effective_date '01/11/2026' is not in YYYY-MM-DD format
  5     E2004     HIRE       REJECTED  -              full_name is empty or longer than 80 characters
  6     E2005     PROMOTE    REJECTED  -              unknown event 'PROMOTE' (expected HIRE/TERMINATE/TRANSFER)
  7     E         HIRE       REJECTED  -              invalid employee_id 'E' (3-20 letters, digits or '-')

WARNING 5 row(s) were rejected - see the report; exit code 1
1
```

Figure 11. *`hr_bad.csv`: five invalid rows and one valid row.*

What it shows. The valid row was still created; the five bad rows were rejected one by one (invalid ID, unknown department, wrong date format, empty name, unknown event). One bad row never stops the run.

```
PS C:\Users\اسماعيل\OneDrive\Desktop\IT_Intern_Projects\employee-account-automation> python account_automation.py --hr data/nope.
>> echo $LASTEXITCODE
ERROR   HR file not found: data/nope.csv
2
```

Figure 12. *Missing HR file (`nope.csv`).*

What it shows. A one-line error message and exit code 2 instead of a Python traceback.

```
PS C:\Users\اسماعيل\OneDrive\Desktop\IT_Intern_Projects\employee-account-automation> Set-Content -Path data/corrupt.json -Value '{"broken'
>> python account_automation.py --hr data/hr_day1.csv --directory data/corrupt.json --as-of 2026-11-01
>> echo $LASTEXITCODE
>> Get-Content data/corrupt.json
ERROR   Directory file data/corrupt.json is corrupt (Invalid control character at: line 1 column 9 (char 8)). It was NOT modified; restore a g
ood copy (data/corrupt.json.bak if it exists) and run again.
2
{"broken
```

Figure 13. *Corrupt `directory.json`.*

What it shows. The tool refuses to continue and states that the file was NOT modified; the file content is unchanged afterwards. A broken file is never silently replaced by an empty one, which would have wiped all accounts.

```
PS C:\Users\ايلياس\OneDrive\Desktop\IT_Intern_Projects\employee-account-automation> python account_automation.py --hr data/hr_day2.csv --as-of
2026-12-20
WARNING Temporary passwords were written to output\new_credentials_2026-12-20_115447.csv - hand them over securely, then delete the file

EMPLOYEE ACCOUNT AUTOMATION REPORT
==================================
Run date : 2026-12-20
Mode     : APPLY
HR file  : hr_day2.csv

SUMMARY
  CREATED    1
  SKIPPED    3
  PURGE_DUE  1
  REJECTED   1

DETAILS
  line  employee  event      status    username      detail
  2     E1005     TRANSFER   SKIPPED   ahassan       already in Finance
  3     E1002     TERMINATE  SKIPPED   salharbi      account already disabled
  4     E1001     HIRE       SKIPPED   eghwiet       already has an active account
  5     E1099     TERMINATE  REJECTED  -             no account exists for employee E1099
  6     E1007     HIRE       CREATED   librahim      department Sales, groups all-staff, crm-access, sales-users
  -     -         PURGE      PURGE_DUE salharbi      disabled longer than 30 days - run with --purge to delete

WARNING 1 row(s) were rejected - see the report; exit code 1
```

Figure 14. *Run on 2026-12-20.*

What it shows. The row E1007 that was deferred on 1 November is now applied: `librahim` is CREATED. `salharbi`, disabled on 2026-11-15, has been disabled for more than 30 days and is reported as PURGE_DUE. This single screenshot proves the deferral, the retention logic and the date-driven design together.

```
PS C:\Users\ايلياس\OneDrive\Desktop\IT_Intern_Projects\employee-account-automation> python account_automation.py --hr data/hr_day2.csv --as-of 2026-12
-20 --purge

EMPLOYEE ACCOUNT AUTOMATION REPORT
==================================
Run date : 2026-12-20
Mode     : APPLY
HR file  : hr_day2.csv

SUMMARY
  PURGED     1
  SKIPPED    4
  REJECTED   1

DETAILS
  line  employee  event      status    username      detail
  2     E1005     TRANSFER   SKIPPED   ahassan       already in Finance
  3     E1002     TERMINATE  SKIPPED   salharbi      account already disabled
  4     E1001     HIRE       SKIPPED   eghwiet       already has an active account
  5     E1099     TERMINATE  REJECTED  -             no account exists for employee E1099
  6     E1007     HIRE       SKIPPED   librahim      already has an active account
  -     -         PURGE      PURGED    salharbi      disabled longer than 30 days

WARNING 1 row(s) were rejected - see the report; exit code 1
```

Figure 14B. *Run on 2026-12-20.*

What it shows. The same command with --purge: salharbi, disabled on 2026-11-15, is now actually deleted (PURGED 1). Earlier runs only reported it as PURGE_DUE.

```
PS C:\Users\من اول\OneDrive\Desktop\IT_Intern_Projects\employee-account-automation> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.6, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\من اول\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\من اول\OneDrive\Desktop\IT_Intern_Projects\employee-account-automation
collected 41 items

tests/test_account_automation.py::test_base_username_first_initial_plus_last_name PASSED [  2%]
tests/test_account_automation.py::test_username_removes_hyphens_apostrophes_and_accents PASSED [  4%]
tests/test_account_automation.py::test_username_from_arabic_name_is_ascii PASSED [  7%]
tests/test_account_automation.py::test_single_word_name_and_length_limit PASSED [  9%]
tests/test_account_automation.py::test_name_without_letters_is_rejected PASSED [ 12%]
tests/test_account_automation.py::test_collisions_get_a_number PASSED [ 14%]
tests/test_account_automation.py::test_collision_suffix_respects_max_length PASSED [ 17%]
tests/test_account_automation.py::test_temp_password_is_strong_and_random PASSED [ 19%]
tests/test_account_automation.py::test_password_is_stored_only_as_a_hash PASSED [ 21%]
tests/test_account_automation.py::test_hire_creates_account_with_department_groups PASSED [ 24%]
tests/test_account_automation.py::test_latin_name_takes_precedence_over_arabic_name PASSED [ 26%]
tests/test_account_automation.py::test_two_people_with_same_username_base PASSED [ 29%]
tests/test_account_automation.py::test_future_hire_is_deferred_then_applied_later PASSED [ 31%]
tests/test_account_automation.py::test_running_same_file_twice_changes_nothing PASSED [ 34%]
tests/test_account_automation.py::test_terminate_disables_and_removes_access PASSED [ 36%]
tests/test_account_automation.py::test_terminate_unknown_employee_is_rejected PASSED [ 39%]
tests/test_account_automation.py::test_terminate_twice_is_skipped PASSED [ 41%]
tests/test_account_automation.py::test_transfer_changes_groups PASSED [ 43%]
tests/test_account_automation.py::test_transfer_of_disabled_account_is_rejected PASSED [ 46%]
tests/test_account_automation.py::test_rehire_reenables_same_username PASSED [ 48%]
tests/test_account_automation.py::test_events_in_one_file_apply_in_date_order PASSED [ 51%]
tests/test_account_automation.py::test_purge_due_is_reported_and_purge_deletes PASSED [ 53%]
tests/test_account_automation.py::test_recently_disabled_account_is_not_purge_due PASSED [ 56%]
tests/test_account_automation.py::test_invalid_rows_are_rejected_but_run_continues[row0-unknown department] PASSED [ 58%]
tests/test_account_automation.py::test_invalid_rows_are_rejected_but_run_continues[row1-YYYY-MM-DD] PASSED [ 60%]
tests/test_account_automation.py::test_invalid_rows_are_rejected_but_run_continues[row2-full_name] PASSED [ 63%]
tests/test_account_automation.py::test_invalid_rows_are_rejected_but_run_continues[row3-unknown event] PASSED [ 65%]
tests/test_account_automation.py::test_invalid_rows_are_rejected_but_run_continues[row4-employee_id] PASSED [ 68%]
tests/test_account_automation.py::test_invalid_rows_are_rejected_but_run_continues[row5-YYYY-MM-DD] PASSED [ 70%]
tests/test_account_automation.py::test_dry_run_changes_nothing PASSED [ 73%]
tests/test_account_automation.py::test_audit_log_report_and_credentials_are_written PASSED [ 75%]
tests/test_account_automation.py::test_second_run_on_same_day_does_not_overwrite_credentials PASSED [ 78%]
tests/test_account_automation.py::test_missing_hr_file_is_fatal PASSED [ 80%]
tests/test_account_automation.py::test_missing_column_is_fatal PASSED [ 82%]
tests/test_account_automation.py::test_corrupt_directory_is_fatal_and_not_overwritten PASSED [ 85%]
tests/test_account_automation.py::test_bad_policy_is_fatal PASSED [ 87%]
tests/test_account_automation.py::test_excel_bom_and_blank_lines_are_handled PASSED [ 90%]
tests/test_account_automation.py::test_backup_of_previous_directory_is_kept PASSED [ 92%]
tests/test_account_automation.py::test_exit_code_0_when_all_rows_ok PASSED [ 95%]
tests/test_account_automation.py::test_exit_code_1_when_a_row_is_rejected PASSED [ 97%]
tests/test_account_automation.py::test_exit_code_2_on_fatal_error PASSED [100%]

============================== 41 passed in 5.36s ==============================
```

Figure 15. *`pytest -v`: 41 passed in 5.36 s.*

What it shows. All automated tests pass on my machine (Python 3.14.6, pytest 9.2.0).

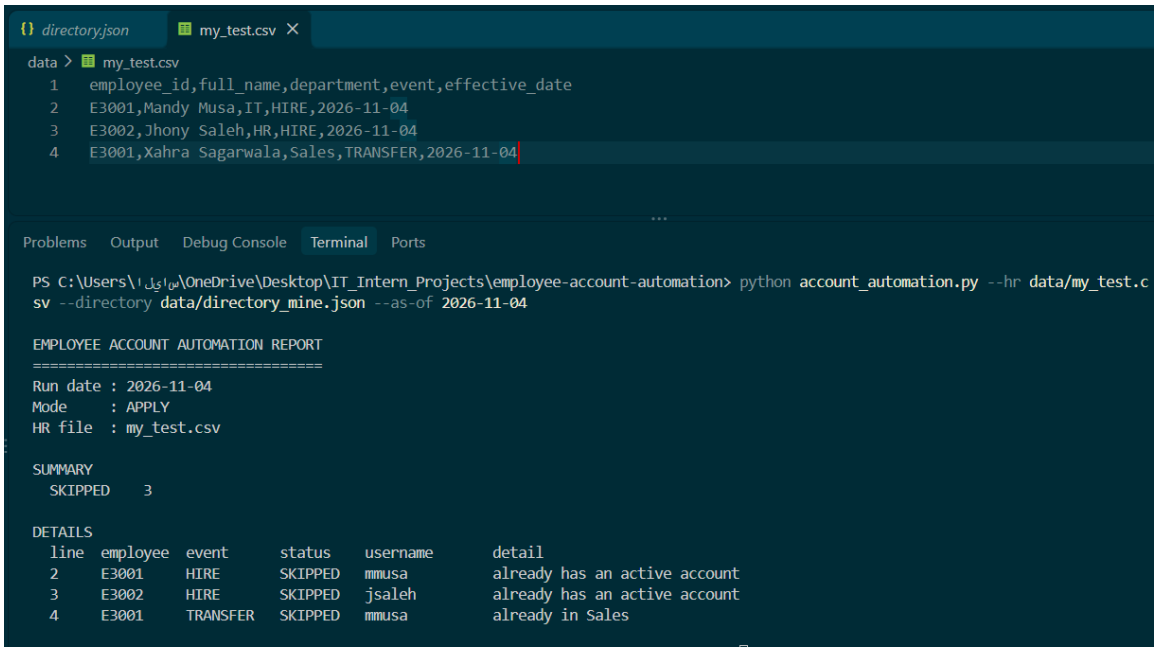


Figure 16. *My own file `my_test.csv` with a separate directory `directory_mine.json`.*

What it shows. I created this input myself (E3001 Mandy Musa, E3002 Jhony Saleh, then a TRANSFER of E3001 to Sales). The re-run shows 3 SKIPPED: E3001 is already in Sales, so the transfer is not applied twice. Line 4 carries a

different name ("Xahra Sagarwala") for the same ID; the tool matches by `employee_id` only and does not complain, which is a documented limitation (Section 8).

Evidence limits: the first run of my_test.csv was not captured; only its re-run is shown.

6. Challenges and what I fixed

My own tests and manual runs found these problems before submission:

#	Problem found	How it was found	Fix
1	A name made only of digits or symbols produced an empty username and was accepted.	Test `test_name_without_letters_is_rejected` failed.	`base_username` now raises RowError unless the name contains at least one letter (line 92).
2	Two runs on the same day overwrote the first report and credentials file, so passwords from the first run were lost.	`test_second_run_on_same_day_does_not_overwrite_credentials` failed.	File names now carry date and time: `report_2026-11-01_113514.txt`.
3	My own test data used 2-character IDs (E1...E9) that the script rightly refused (3-20 characters).	Many tests failed at once with REJECTED.	Test IDs changed to E101...; the rule was kept.
4	The corrupt-directory message told the user to restore from `.bak` even when none exists.	Reading the message during the TC-08 test.	Message now says "restore a good copy (<file>.bak if it exists)".
5	Transliteration happened after Unicode normalisation, so some Arabic letters were lost.	Arabic-name test produced a shorter username than expected.	Arabic letters are mapped first, then accents are stripped (lines 76-81).
6	Running the tool on my own CSV failed with "HR file not found" and "missing required column(s)".	Real use on Windows: the file had not been saved / was still empty in VS Code.	No code change needed - the messages named the exact problem; this confirmed that the error handling guides the user.

7. Originality and creative features

- **Arabic-aware identity handling.** Names in Arabic script are transliterated into ASCII usernames; HR can override with `latin_name`. This reflects my own working environment and is not part of a typical tutorial script.
- **Deferred events.** Future-dated rows are kept and applied on the run whose date reaches them (E1007 in Figure 14).
- **Time machine for testing.** `--as-of` lets me simulate days or months and test the 30-day retention rule in seconds.
- **Complete joiner / mover / leaver cycle with re-hire.** A returning employee gets the same username back instead of a duplicate.
- **Defence in depth for data:** atomic save, `.bak` backup, corrupt files never overwritten, credentials file with restricted permissions, passwords only as salted hashes.
- **Three outputs per run** (audit log, report, credentials) and **machine-readable exit codes** so the tool can be scheduled.
- **Clean replacement point.** `DirectoryStore` is the only class that touches the directory; the HR parsing, validation and rules would stay unchanged for a real Active Directory.

8. Limitations and future work

- **Simulated directory.** It does not create real Windows or Linux accounts. Next step: implement a `DirectoryStore` that calls PowerShell `New-ADUser` / `Disable-ADAccount` or LDAP.
- **Password iterations.** 200,000 PBKDF2 rounds is above the NIST minimum but below OWASP's current 600,000 recommendation; production use needs a higher value (or Argon2/bcrypt).
- **Credential delivery.** Temporary passwords are written to a local CSV. A real system should deliver them through a secrets manager or directly to the manager by a secure channel.
- **Name mismatch is not detected.** Matching is by `employee_id` only (seen in Figure 16); a warning when the name differs would catch data-entry errors.
- **No locking.** Two simultaneous runs could race on the same directory file; a lock file would prevent that.
- **Scheduling and notifications.** The exit codes already allow Task Scheduler or cron; sending the report by e-mail is a natural extension.

9. Time log

Activity	Hours
Research and reading (CIS, NIST, OWASP, Python documentation)	7
Designing the idea and understanding the script (rules, data model, line-by-line study)	13
Running, testing and taking screenshots	4
Writing and reviewing the report (with AI assistance for drafting, reviewed by me)	11
Total	**35**

The total of 35 hours is inside the 30-35 hour range of the task.

10. Conclusion

The project turns a repetitive and risky help-desk job into a repeatable, tested and traceable procedure. The tool handled the normal flow, repeated runs, department changes, leavers, bad input, a missing file, a corrupt directory and a date-driven purge, and 41 automated tests confirm the behaviour. I learned that most of the work in automation is not the happy path but the edge cases: duplicates, order of events, partial failures and what must never be overwritten. The next step would be to connect the same logic to a real directory service.

Sources

- CIS Controls v8, Safeguard 5.3 Disable Dormant Accounts - https://controls-assessment-specification.readthedocs.io/en/latest/control-5/control-5.3.html
- NIST SP 800-53 Rev. 5, AC-2(3) Disable Accounts - https://csf.tools/reference/nist-sp-800-53/r5/ac/ac-2/ac-2-3/
- OWASP Password Storage Cheat Sheet - https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- NIST SP 800-63B, Digital Identity Guidelines: Authentication and Lifecycle Management (2017) - https://pages.nist.gov/800-63-3/sp800-63b.html
- Python Standard Library documentation: hashlib, secrets, csv, argparse, tempfile, os - https://docs.python.org/3/library/

Source code and run guide: public GitHub repository (link in the submission form).

Appendix A - Complete source code (account_automation.py)

Non-ASCII characters (the Arabic transliteration table) are shown as \uXXXX escapes so that every character prints correctly. The file in the repository contains the real characters.

```
#!/usr/bin/env python3
"""Employee Account Lifecycle Automation.


Reads a CSV of HR events (HIRE, TERMINATE, TRANSFER) and keeps a (simulated) user
directory in step with it:


* HIRE      -> create an account (or re-enable a disabled one), generate a unique
               username, assign groups from the department policy, issue a temporary password
* TERMINATE -> disable the account, remove its groups, invalidate the password
* TRANSFER  -> move the account to the new department's groups


Every run writes an audit log and a report. The script is idempotent: running the same
HR file twice changes nothing the second time. Use --dry-run to preview a run.


Only the Python standard library is used. The directory is a JSON file, so the script can be
tested without administrator rights; the DirectoryStore class is the single place that would
be replaced by real calls (net user, useradd, LDAP) in a production deployment.
"""
import argparse
import csv
import hashlib
import json
import logging
import os
import re
import secrets
import shutil
import string
import sys
import tempfile
import unicodedata
from dataclasses import dataclass
from datetime import date, datetime, timedelta


EXIT_OK, EXIT_REJECTED, EXIT_FATAL = 0, 1, 2
REQUIRED_COLUMNS = ["employee_id", "full_name", "department", "event", "effective_date"]
EVENTS = ("HIRE", "TERMINATE", "TRANSFER")
EMPLOYEE_ID_RE = re.compile(r"^[A-Za-z0-9-]{3,20}$")
USERNAME_MAX = 20
PBKDF2_ROUNDS = 200_000


log = logging.getLogger("account_automation")


# Approximate Arabic -> Latin transliteration so that HR files written in Arabic still
# produce ASCII usernames. A latin_name column in the HR file always takes precedence.
AR_MAP = {
    "\u0627": "a", "\u0623": "a", "\u0625": "i", "\u0622": "a", "\u0628": "b", "\u062a": "t", "\u062b": "th", "\u062c": "j",
"\u062d": "h",
    "\u062e": "kh", "\u062f": "d", "\u0630": "dh", "\u0631": "r", "\u0632": "z", "\u0633": "s", "\u0634": "sh", "\u0635": "s",
"\u0636": "d",
    "\u0637": "t", "\u0638": "z", "\u0639": "a", "\u063a": "gh", "\u0641": "f", "\u0642": "q", "\u0643": "k", "\u0644": "l",
"\u0645": "m",
    "\u0646": "n", "\u0647": "h", "\u0648": "w", "\u064a": "y", "\u0649": "a", "\u0629": "a", "\u0621": "", "\u0626": "y",
"\u0624": "w",
}



class RowError(Exception):
    """A single HR row is invalid; the row is rejected and the run continues."""



class FatalError(Exception):
    """The run cannot continue (missing file, corrupt directory, bad policy)."""



@dataclass
class Result:
    line: int
    employee_id: str
    action: str
    status: str
    username: str = ""
    detail: str = ""
```

```
# ---------------------------------------------------------------------------- names
def to_ascii_words(name):
    """Return the lower-case ASCII words of a name (Arabic letters are transliterated)."""
    out = []
    for original in name:
        if original in AR_MAP:                                  # map Arabic before normalising
            out.append(AR_MAP[original])
            continue
        for ch in unicodedata.normalize("NFKD", original):          # strips accents: Jos\u00e9 -> jose
            if unicodedata.combining(ch):
                continue
            if ch.isascii() and ch.isalnum():
                out.append(ch.lower())
            elif ch.isspace():
                out.append(" ")                                 # other characters (-, ', digits-less symbols) are dropped
    return "".join(out).split()



def base_username(name, max_len=USERNAME_MAX):
    words = to_ascii_words(name)
    if not any(ch.isalpha() for w in words for ch in w):
        raise RowError("name has no usable letters for a username")
    first, last = words[0], words[-1]
    return (first[0] + last if len(words) > 1 else first)[:max_len]



def unique_username(base, taken):
    """base, base2, base3, ... - names of disabled accounts stay taken (audit trail)."""
    if base not in taken:
        return base
    n = 2
    while "%s%d" % (base[:USERNAME_MAX - len(str(n))], n) in taken:
        n += 1
    return "%s%d" % (base[:USERNAME_MAX - len(str(n))], n)



# ---------------------------------------------------------------------------- passwords
def new_temp_password(length=14):
    alphabet = string.ascii_letters + string.digits + "!@#$%&*"
    while True:
        pw = "".join(secrets.choice(alphabet) for _ in range(length))
        if (any(c.islower() for c in pw) and any(c.isupper() for c in pw)
                and any(c.isdigit() for c in pw) and any(c in "!@#$%&*" for c in pw)):
            return pw



def hash_password(pw):
    salt = secrets.token_hex(8)
    digest = hashlib.pbkdf2_hmac("sha256", pw.encode(), salt.encode(), PBKDF2_ROUNDS).hex()
    return "pbkdf2_sha256$%d$%s$%s" % (PBKDF2_ROUNDS, salt, digest)



# ---------------------------------------------------------------------------- directory
class DirectoryStore:
    """Simulated user directory kept in a JSON file (replace this class for a real system)."""


    def __init__(self, path):
        self.path = path
        self.data = {"version": 1, "users": {}}


    def load(self):
        if not os.path.exists(self.path):
            log.info("Directory %s does not exist yet - starting with an empty directory", self.path)
            return
        try:
            with open(self.path, encoding="utf-8") as f:
                data = json.load(f)
            if not isinstance(data, dict) or not isinstance(data.get("users"), dict):
                raise ValueError("missing 'users' object")
        except (json.JSONDecodeError, ValueError) as exc:
            raise FatalError("Directory file %s is corrupt (%s). It was NOT modified; restore a "
                             "good copy (%s.bak if it exists) and run again." % (self.path, exc, self.path))
        self.data = data


    def save(self):
        """Write atomically and keep one backup of the previous version."""
        folder = os.path.dirname(os.path.abspath(self.path))
        os.makedirs(folder, exist_ok=True)
        if os.path.exists(self.path):
            shutil.copy2(self.path, self.path + ".bak")
        fd, tmp = tempfile.mkstemp(dir=folder, suffix=".tmp")
        try:
```

```
            with os.fdopen(fd, "w", encoding="utf-8") as f:
                json.dump(self.data, f, indent=2, ensure_ascii=False, sort_keys=True)
            os.replace(tmp, self.path)
        except OSError:
            if os.path.exists(tmp):
                os.remove(tmp)
            raise

    @property
    def users(self):
        return self.data["users"]

    def find_by_employee(self, employee_id):
        for username, u in self.users.items():
            if u["employee_id"] == employee_id:
                return username, u
        return None, None



# ---------------------------------------------------------------------- policy
def load_policy(path):
    try:
        with open(path, encoding="utf-8") as f:
            pol = json.load(f)
    except FileNotFoundError:
        raise FatalError("Policy file not found: %s" % path)
    except json.JSONDecodeError as exc:
        raise FatalError("Policy file %s is not valid JSON (%s)" % (path, exc))
    if not isinstance(pol.get("departments"), dict) or not pol["departments"]:
        raise FatalError("Policy file %s must define a non-empty 'departments' object" % path)
    pol.setdefault("default_groups", [])
    pol.setdefault("retention_days", 30)
    pol["_dept_lookup"] = {k.lower(): k for k in pol["departments"]}
    return pol



def groups_for(policy, department):
    return sorted(set(policy["default_groups"]) | set(policy["departments"][department]))



# ---------------------------------------------------------------------- HR input
def read_hr_file(path):
    """Return a list of (line_number, row_dict). Raises FatalError for file-level problems."""
    if not os.path.exists(path):
        raise FatalError("HR file not found: %s" % path)
    try:
        with open(path, newline="", encoding="utf-8-sig") as f:   # utf-8-sig: Excel adds a BOM
            reader = csv.DictReader(f)
            header = [h.strip() for h in (reader.fieldnames or [])]
            missing = [c for c in REQUIRED_COLUMNS if c not in header]
            if missing:
                raise FatalError("HR file %s is missing required column(s): %s" % (path, ", ".join(missing)))
            rows = []
            for row in reader:
                clean = {(k or "").strip(): (v or "").strip() for k, v in row.items()}
                if not any(clean.values()):
                    continue                                    # skip blank lines
                rows.append((reader.line_num, clean))
            return rows
    except UnicodeDecodeError:
        raise FatalError("HR file %s is not valid UTF-8; save it as 'CSV UTF-8' from Excel" % path)



def validate_row(row, policy):
    employee_id = row["employee_id"]
    if not EMPLOYEE_ID_RE.match(employee_id):
        raise RowError("invalid employee_id %r (3-20 letters, digits or '-')" % employee_id)
    if not row["full_name"] or len(row["full_name"]) > 80:
        raise RowError("full_name is empty or longer than 80 characters")
    event = row["event"].upper()
    if event not in EVENTS:
        raise RowError("unknown event %r (expected %s)" % (row["event"], "/".join(EVENTS)))
    try:
        eff = datetime.strptime(row["effective_date"], "%Y-%m-%d").date()
    except ValueError:
        raise RowError("effective_date %r is not in YYYY-MM-DD format" % row["effective_date"])
    dept = policy["_dept_lookup"].get(row["department"].lower())
    if dept is None:
        raise RowError("unknown department %r (policy knows: %s)"
                       % (row["department"], ", ".join(sorted(policy["departments"]))))
    return {"employee_id": employee_id, "full_name": row["full_name"], "event": event,
            "effective": eff, "department": dept, "latin_name": row.get("latin_name", "")}



# ---------------------------------------------------------------------- the actions
def apply_event(ev, store, policy, as_of, credentials):
```

```
    """Apply one validated event to the directory. Returns (status, username, detail)."""
    username, user = store.find_by_employee(ev["employee_id"])
    action, dept = ev["event"], ev["department"]


    if action == "HIRE":
        if user and user["status"] == "active":
            return "SKIPPED", username, "already has an active account"
        pw = new_temp_password()
        if user:                                                # re-hire of a disabled account
            user.update(status="active", department=dept, groups=groups_for(policy, dept),
                        disabled_on=None, password_hash=hash_password(pw), must_change_password=True)
            credentials.append((username, pw))
            return "ENABLED", username, "re-hire: account re-enabled in %s" % dept
        base = base_username(ev["latin_name"] or ev["full_name"])
        username = unique_username(base, set(store.users))
        store.users[username] = {
            "employee_id": ev["employee_id"], "full_name": ev["full_name"], "department": dept,
            "status": "active", "groups": groups_for(policy, dept), "created": as_of.isoformat(),
            "disabled_on": None, "password_hash": hash_password(pw), "must_change_password": True}
        credentials.append((username, pw))
        note = "" if username == base else " (%s was taken)" % base
        return "CREATED", username, "department %s, groups %s%s" % (dept, ", ".join(store.users[username]["groups"]), note)


    if user is None:
        raise RowError("no account exists for employee %s" % ev["employee_id"])


    if action == "TERMINATE":
        if user["status"] == "disabled":
            return "SKIPPED", username, "account already disabled"
        user.update(status="disabled", groups=[], disabled_on=ev["effective"].isoformat(),
                    password_hash=None, must_change_password=False)
        return "DISABLED", username, "groups removed, password invalidated"


    if action == "TRANSFER":
        if user["status"] != "active":
            raise RowError("cannot transfer: account %s is disabled" % username)
        if user["department"] == dept:
            return "SKIPPED", username, "already in %s" % dept
        old = user["department"]
        user.update(department=dept, groups=groups_for(policy, dept))
        return "UPDATED", username, "moved %s -> %s, groups %s" % (old, dept, ", ".join(user["groups"]))
    raise RowError("unsupported event %s" % action)              # not reachable after validation



def purge_candidates(store, policy, as_of):
    limit = as_of - timedelta(days=int(policy["retention_days"]))
    return sorted(u for u, rec in store.users.items()
                  if rec["status"] == "disabled" and rec.get("disabled_on")
                  and date.fromisoformat(rec["disabled_on"]) <= limit)



# ---------------------------------------------------------------------------- the run
def run(hr_path, dir_path, policy_path, out_dir, as_of, dry_run=False, purge=False):
    """Process one HR file. Returns (results, credentials). Raises FatalError."""
    policy = load_policy(policy_path)
    rows = read_hr_file(hr_path)
    store = DirectoryStore(dir_path)
    store.load()


    results, credentials, events = [], [], []
    for line, row in rows:
        try:
            ev = validate_row(row, policy)
            ev["line"] = line
            events.append(ev)
        except RowError as exc:
            results.append(Result(line, row.get("employee_id", ""), row.get("event", "").upper(), "REJECTED", "", str(exc)))


    events.sort(key=lambda e: (e["effective"], e["line"]))        # apply in date order
    for ev in events:
        if ev["effective"] > as_of:
            results.append(Result(ev["line"], ev["employee_id"], ev["event"], "DEFERRED", "",
                                  "effective %s is after run date %s" % (ev["effective"], as_of)))
            continue
        try:
            status, username, detail = apply_event(ev, store, policy, as_of, credentials)
            results.append(Result(ev["line"], ev["employee_id"], ev["event"], status, username, detail))
        except RowError as exc:
            results.append(Result(ev["line"], ev["employee_id"], ev["event"], "REJECTED", "", str(exc)))


    for username in purge_candidates(store, policy, as_of):
        if purge:
            del store.users[username]
            results.append(Result(0, "", "PURGE", "PURGED", username, "disabled longer than %s days" %
policy["retention_days"]))
        else:
            results.append(Result(0, "", "PURGE", "PURGE_DUE", username,
```

```
                                "disabled longer than %s days - run with --purge to delete" % policy["retention_days"]))

    results.sort(key=lambda r: (r.line == 0, r.line))
    if not dry_run:
        write_outputs(store, results, credentials, hr_path, out_dir, as_of)
    return results, credentials



def write_outputs(store, results, credentials, hr_path, out_dir, as_of):
    os.makedirs(out_dir, exist_ok=True)
    stamp = "%s_%s" % (as_of.isoformat(), datetime.now().strftime("%H%M%S"))   # never overwrite an earlier run
    changed = any(r.status in ("CREATED", "ENABLED", "DISABLED", "UPDATED", "PURGED") for r in results)
    if changed or not os.path.exists(store.path):
        store.save()
    with open(os.path.join(out_dir, "audit.log"), "a", encoding="utf-8") as f:
        for r in results:
            f.write("%s | %-9s | %-10s | %-8s | %-14s | %s\n" % (
                datetime.now().isoformat(timespec="seconds"), r.status, r.action, r.employee_id or "-", r.username or "-",
r.detail))
    with open(os.path.join(out_dir, "report_%s.txt" % stamp), "w", encoding="utf-8") as f:
        f.write(build_report(results, hr_path, as_of, dry_run=False))
    if credentials:
        path = os.path.join(out_dir, "new_credentials_%s.csv" % stamp)
        with open(path, "w", newline="", encoding="utf-8") as f:
            w = csv.writer(f)
            w.writerow(["username", "temporary_password"])
            w.writerows(credentials)
        try:
            os.chmod(path, 0o600)
        except OSError:
            pass
        log.warning("Temporary passwords were written to %s - hand them over securely, then delete the file", path)



def build_report(results, hr_path, as_of, dry_run):
    counts = {}
    for r in results:
        counts[r.status] = counts.get(r.status, 0) + 1
    lines = ["EMPLOYEE ACCOUNT AUTOMATION REPORT", "=" * 34,
             "Run date : %s" % as_of, "Mode     : %s" % ("DRY RUN (nothing was changed)" if dry_run else "APPLY"),
             "HR file  : %s" % os.path.basename(hr_path), "", "SUMMARY"]
    for status in ("CREATED", "ENABLED", "UPDATED", "DISABLED", "PURGED", "SKIPPED", "DEFERRED", "PURGE_DUE", "REJECTED"):
        if status in counts:
            lines.append("  %-10s %d" % (status, counts[status]))
    lines += ["", "DETAILS", "  %-5s %-9s %-10s %-9s %-14s %s" % ("line", "employee", "event", "status", "username", "detail")]
    for r in results:
        lines.append("  %-5s %-9s %-10s %-9s %-14s %s" % (r.line or "-", r.employee_id or "-", r.action, r.status, r.username
or "-", r.detail))
    return "\n".join(lines) + "\n"



# ---------------------------------------------------------------------------- CLI
def parse_args(argv=None):
    p = argparse.ArgumentParser(description="Apply HR events (HIRE/TERMINATE/TRANSFER) to a simulated user directory.")
    p.add_argument("--hr", required=True, help="HR events CSV file")
    p.add_argument("--directory", default="data/directory.json", help="user directory JSON file (default:
data/directory.json)")
    p.add_argument("--policy", default="data/policy.json", help="department/group policy JSON (default: data/policy.json)")
    p.add_argument("--out-dir", default="output", help="folder for audit log, report and credentials (default: output)")
    p.add_argument("--as-of", help="run date YYYY-MM-DD (default: today); events after this date are deferred")
    p.add_argument("--dry-run", action="store_true", help="show what would happen, change nothing")
    p.add_argument("--purge", action="store_true", help="delete accounts disabled longer than the retention period")
    return p.parse_args(argv)



def main(argv=None):
    for stream in (sys.stdout, sys.stderr):                      # Arabic names on Windows consoles
        try:
            stream.reconfigure(encoding="utf-8", errors="replace")
        except (AttributeError, OSError):
            pass
    args = parse_args(argv)
    logging.basicConfig(level=logging.INFO, format="%(levelname)-7s %(message)s")
    try:
        as_of = datetime.strptime(args.as_of, "%Y-%m-%d").date() if args.as_of else date.today()
    except ValueError:
        log.error("--as-of must be YYYY-MM-DD")
        return EXIT_FATAL
    try:
        results, _ = run(args.hr, args.directory, args.policy, args.out_dir, as_of, args.dry_run, args.purge)
    except FatalError as exc:
        log.error("%s", exc)
        return EXIT_FATAL
    except OSError as exc:
        log.error("File system error: %s", exc)
        return EXIT_FATAL
```

```
    print()
    print(build_report(results, args.hr, as_of, args.dry_run))
    rejected = sum(1 for r in results if r.status == "REJECTED")
    if rejected:
        log.warning("%d row(s) were rejected - see the report; exit code 1", rejected)
        return EXIT_REJECTED
    return EXIT_OK



if __name__ == "__main__":
    sys.exit(main())
```

Appendix B - List of automated tests

36 test functions (the validation test is parametrised and runs several cases, which gives 41 results in total).

#	Test function
1	test_base_username_first_initial_plus_last_name
2	test_username_removes_hyphens_apostrophes_and_accents
3	test_username_from_arabic_name_is_ascii
4	test_single_word_name_and_length_limit
5	test_name_without_letters_is_rejected
6	test_collisions_get_a_number
7	test_collision_suffix_respects_max_length
8	test_temp_password_is_strong_and_random
9	test_password_is_stored_only_as_a_hash
10	test_hire_creates_account_with_department_groups
11	test_latin_name_takes_precedence_over_arabic_name
12	test_two_people_with_same_username_base
13	test_future_hire_is_deferred_then_applied_later
14	test_running_same_file_twice_changes_nothing
15	test_terminate_disables_and_removes_access
16	test_terminate_unknown_employee_is_rejected
17	test_terminate_twice_is_skipped
18	test_transfer_changes_groups
19	test_transfer_of_disabled_account_is_rejected
20	test_rehire_reenables_same_username
21	test_events_in_one_file_apply_in_date_order
22	test_purge_due_is_reported_and_purge_deletes
23	test_recently_disabled_account_is_not_purge_due
24	test_invalid_rows_are_rejected_but_run_continues
25	test_dry_run_changes_nothing
26	test_audit_log_report_and_credentials_are_written
27	test_second_run_on_same_day_does_not_overwrite_credentials
28	test_missing_hr_file_is_fatal
29	test_missing_column_is_fatal
30	test_corrupt_directory_is_fatal_and_not_overwritten

#	Test function
31	test_bad_policy_is_fatal
32	test_excel_bom_and_blank_lines_are_handled
33	test_backup_of_previous_directory_is_kept
34	test_exit_code_0_when_all_rows_ok
35	test_exit_code_1_when_a_row_is_rejected
36	test_exit_code_2_on_fatal_error

Appendix C - Policy and sample HR files

```
{
  "retention_days": 30,
  "default_groups": ["all-staff"],
  "departments": {
    "IT": ["it-staff", "remote-access", "admin-tools"],
    "HR": ["hr-users", "payroll-readonly"],
    "Finance": ["finance-users", "erp-access"],
    "Sales": ["sales-users", "crm-access"]
  }
}
```

data/policy.json

```
employee_id,full_name,department,event,effective_date,latin_name
E1001,Elyas Ghwiet,IT,HIRE,2026-11-01,
E1002,Sara Al-Harbi,HR,HIRE,2026-11-01,
E1003,\u0645\u062d\u0645\u062f \u0627\u0644\u062d\u0631\u0628\u064a,Finance,HIRE,2026-11-01,Mohammed Alharbi
E1004,\u0646\u0648\u0631\u0629 \u0627\u0644\u0633\u0627\u0644\u0645,HR,HIRE,2026-11-01,
E1005,Ali Hassan,IT,HIRE,2026-11-01,
E1006,Ahmed Hassan,Finance,HIRE,2026-11-01,
E1007,Layla Ibrahim,Sales,HIRE,2026-11-20,
```

data/hr_day1.csv

```
employee_id,full_name,department,event,effective_date,latin_name
E1005,Ali Hassan,Finance,TRANSFER,2026-11-10,
E1002,Sara Al-Harbi,HR,TERMINATE,2026-11-12,
E1001,Elyas Ghwiet,IT,HIRE,2026-11-01,
E1099,Unknown Person,IT,TERMINATE,2026-11-12,
E1007,Layla Ibrahim,Sales,HIRE,2026-11-20,
```

data/hr_day2.csv

```
employee_id,full_name,department,event,effective_date,latin_name
E2001,Omar Khaled,IT,HIRE,2026-11-01,
E2002,Huda Ali,Legal,HIRE,2026-11-01,
E2003,Khalid Nasser,IT,HIRE,01/11/2026,
E2004,,IT,HIRE,2026-11-01,
E2005,Mona Said,IT,PROMOTE,2026-11-01,
E,Zed Abc,IT,HIRE,2026-11-01,
```

data/hr_bad.csv